

# Projeto 2 – Sistema de Gerenciamento de Biblioteca

O Projeto 2 será adaptado para o desenvolvimento de uma aplicação web completa para gerenciar uma **biblioteca**, com **API backend em Node.js/Express**, banco relacional com **Sequelize**, autenticação com **JWT**, documentação com **Swagger** e frontend em **React**.

## Funcionalidades detalhadas do Sistema de Biblioteca

O sistema deverá possuir **3 tipos de usuários**:

1. **Administrador**
  2. **Bibliotecário**
  3. **Leitor/Aluno**
- 

### 1. Administrador

O administrador possui acesso completo ao sistema.

#### O que pode fazer:

- Cadastrar usuários do sistema;
  - Editar usuários;
  - Excluir usuários;
  - Definir o tipo de usuário:
    - Administrador;
    - Bibliotecário;
    - Leitor.
  - Cadastrar livros;
  - Editar livros;
  - Excluir livros;
  - Visualizar todos os livros;
  - Visualizar todos os leitores;
  - Visualizar todos os empréstimos;
  - Realizar empréstimos;
  - Registrar devoluções;
  - Acessar relatórios gerais do sistema.
- 

### 2. Bibliotecário

O bibliotecário será responsável pela operação da biblioteca.

### **O que pode fazer:**

- Cadastrar livros;
- Editar livros;
- Visualizar livros;
- Buscar livros por título, autor, categoria ou disponibilidade;
- Cadastrar leitores;
- Editar leitores;
- Visualizar leitores;
- Registrar empréstimos;
- Registrar devoluções;
- Consultar histórico de empréstimos;
- Verificar livros atrasados.

### **O que não pode fazer:**

- Excluir usuários administradores;
  - Alterar permissões de usuários;
  - Excluir dados importantes sem autorização.
- 

## **3. Leitor/Aluno**

O leitor terá acesso limitado ao sistema.

### **O que pode fazer:**

- Fazer login;
- Visualizar livros disponíveis;
- Buscar livros por título, autor ou categoria;
- Consultar seus próprios empréstimos;
- Verificar data prevista de devolução;
- Consultar seu histórico de empréstimos.

### **O que não pode fazer:**

- Cadastrar livros;
  - Editar livros;
  - Excluir livros;
  - Cadastrar outros usuários;
  - Registrar empréstimos;
  - Registrar devoluções.
- 

## **1. Autenticação e Controle de Acesso**

O sistema deverá possuir login com **JWT**.

Cada usuário deverá acessar o sistema conforme seu perfil.

**Regras:**

- Usuário sem login não acessa o sistema administrativo.
  - Administrador acessa todas as funcionalidades.
  - Bibliotecário acessa livros, leitores e empréstimos.
  - Leitor acessa apenas consulta de livros e seus próprios empréstimos.
- 

## 2. Gerenciamento de Livros

Cada livro deverá possuir, no mínimo:

- Título;
- Autor;
- Editora;
- Ano de publicação;
- Categoria;
- ISBN;
- Quantidade total;
- Quantidade disponível;
- Status: disponível ou indisponível.

**Funcionalidades:**

- Cadastrar livro;
  - Listar livros;
  - Buscar livro;
  - Filtrar por categoria;
  - Filtrar por disponibilidade;
  - Editar livro;
  - Excluir livro.
- 

## 3. Gerenciamento de Leitores

Cada leitor deverá possuir:

- Nome;
- CPF ou RA;
- E-mail;
- Telefone;
- Endereço;
- Status: ativo ou inativo.

**Funcionalidades:**

- Cadastrar leitor;
- Listar leitores;
- Buscar leitor por nome, CPF ou RA;
- Editar leitor;

- Inativar leitor;
  - Consultar histórico de empréstimos do leitor.
- 

## 4. Gerenciamento de Empréstimos

Cada empréstimo deverá possuir:

- Leitor;
- Livro;
- Data do empréstimo;
- Data prevista de devolução;
- Data real de devolução;
- Status:
  - Em aberto;
  - Devolvido;
  - Atrasado.

### Regras obrigatórias:

- Um livro só pode ser emprestado se houver quantidade disponível.
  - Ao realizar empréstimo, a quantidade disponível do livro deve diminuir.
  - Ao registrar devolução, a quantidade disponível deve aumentar.
  - Um leitor inativo não pode realizar empréstimo.
  - O sistema deve indicar empréstimos atrasados.
  - O leitor só pode visualizar seus próprios empréstimos.
- 

## 5. Busca e Filtros

O sistema deverá permitir:

- Buscar livro por título;
  - Buscar livro por autor;
  - Buscar livro por categoria;
  - Buscar livro por ISBN;
  - Filtrar livros disponíveis;
  - Buscar leitor por nome;
  - Buscar empréstimos por status;
  - Buscar empréstimos por data;
  - Buscar empréstimos por leitor.
- 

## Quantidade de usuários no sistema

O sistema deverá permitir o cadastro de **vários usuários**, mas obrigatoriamente deve conter pelo menos:

- 1 usuário Administrador;

- 1 usuário Bibliotecário;
- 2 usuários Leitores.

Durante a apresentação, o grupo deverá demonstrar o funcionamento com esses perfis diferentes.

## **Funcionalidades obrigatórias**

### **1. Gerenciamento de Livros**

- Cadastrar livros;
- Listar livros;
- Editar informações do livro;
- Excluir livros;
- Buscar livros por título, autor, categoria ou disponibilidade.

### **2. Gerenciamento de Usuários/Leitores**

- Cadastrar leitores;
- Listar leitores;
- Editar dados dos leitores;
- Excluir leitores;
- Buscar leitor por nome ou CPF.

### **3. Empréstimos de Livros**

- Registrar empréstimo;
- Associar empréstimo a um leitor;
- Associar empréstimo a um ou mais livros;
- Informar data de empréstimo;
- Informar data prevista de devolução;
- Registrar devolução;
- Controlar se o livro está disponível ou emprestado.

### **4. Autenticação com JWT**

- Cadastro de usuário do sistema;
- Login;
- Geração de token JWT;
- Proteção das rotas da API;
- Apenas usuários autenticados poderão cadastrar, editar, excluir ou consultar dados.

### **5. Documentação com Swagger**

A API deverá conter documentação acessível por uma rota como:

`/api-docs`

A documentação deve apresentar:

- Rotas de autenticação;
- Rotas de livros;

- Rotas de leitores;
- Rotas de empréstimos;
- Métodos GET, POST, PUT e DELETE;
- Parâmetros;
- Corpo das requisições;
- Respostas esperadas.

## **Tecnologias obrigatórias**

### **Backend**

- Node.js;
- Express;
- Sequelize;
- PostgreSQL ou MySQL;
- JWT;
- Swagger.

### **Frontend**

- React;
- Consumo completo da API;
- Telas para login, livros, leitores e empréstimos.

## **Requisitos de avaliação**

### **Requisito 1 – API REST**

Criar uma API que implemente corretamente as funcionalidades de livros, leitores e empréstimos.

### **Requisito 2 – Frontend React**

Criar um frontend que consuma todos os métodos da API, permitindo que o usuário execute todas as funcionalidades do sistema.

### **Requisito 3 – Swagger**

Documentar a API com Swagger.

### **Requisito 4 – JWT**

Implementar autenticação obrigatória com JWT.

## **Entrega**

O grupo deverá entregar:

- Código-fonte do backend;
- Código-fonte do frontend;
- Banco de dados configurado;

- Documentação Swagger funcionando;
- Projeto funcionando durante a apresentação.

## Rubrica de Avaliação – Projeto 2

### Sistema de Gerenciamento de Biblioteca

**Valor Total: 10,0 pontos**

A avaliação será realizada durante a apresentação e demonstração do sistema por todos os integrantes do grupo.

| <b>Critério</b>                               | <b>Descrição</b>                                                                                               | <b>Pontuação</b> |
|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------|------------------|
| <b>1. Modelagem do Banco de Dados</b>         | Estrutura adequada das tabelas, relacionamentos, chaves estrangeiras e normalização dos dados.                 | <b>1,0</b>       |
| <b>2. API REST (CRUD Livros)</b>              | Implementação correta dos endpoints de cadastro, consulta, atualização e exclusão de livros.                   | <b>1,0</b>       |
| <b>3. API REST (CRUD Leitores)</b>            | Implementação correta dos endpoints de cadastro, consulta, atualização e exclusão de leitores.                 | <b>1,0</b>       |
| <b>4. API REST (Empréstimos e Devoluções)</b> | Implementação correta dos empréstimos, devoluções e atualização automática da disponibilidade dos livros.      | <b>1,5</b>       |
| <b>5. Regras de Negócio</b>                   | Controle de estoque/disponibilidade, empréstimos em atraso, bloqueios e validações implementadas corretamente. | <b>1,0</b>       |
| <b>6. Autenticação JWT</b>                    | Cadastro, login, geração de token, proteção de rotas e controle de acesso por perfil de usuário.               | <b>1,0</b>       |
| <b>7. Documentação Swagger</b>                | Documentação completa e funcional contendo todas as rotas da API.                                              | <b>0,5</b>       |
| <b>8. Frontend React</b>                      | Interface funcional consumindo todos os endpoints da API.                                                      | <b>1,0</b>       |
| <b>9. Busca e Filtros</b>                     | Implementação dos filtros e pesquisas exigidos no projeto.                                                     | <b>0,5</b>       |
| <b>10. Apresentação e Domínio do Projeto</b>  | Participação dos integrantes, conhecimento do código e capacidade de responder às perguntas.                   | <b>1,5</b>       |
| <b>Total: 10,0 pontos</b>                     |                                                                                                                |                  |

## Critérios de Desconto

### Banco de Dados (-0,25 a -1,0)

- Relacionamentos incorretos;
- Falta de tabelas obrigatórias;
- Estrutura inconsistente.

## **API (-0,25 a -2,0)**

- Endpoints não funcionando;
- Métodos HTTP incorretos;
- Falta de tratamento de erros.

## **JWT (-0,25 a -1,0)**

- Rotas sem proteção;
- Token inválido;
- Ausência de controle de permissões.

## **Swagger (-0,25 a -0,5)**

- Rotas não documentadas;
- Documentação incompleta.

## **React (-0,25 a -1,0)**

- Telas não funcionais;
- Consumo incorreto da API;
- Operações CRUD incompletas.

## **Apresentação (-0,25 a -1,5)**

- Integrantes não participam;
- Não sabem explicar o funcionamento;
- Não conseguem localizar ou explicar o código.

## **Bônus (até +1,0 ponto)**

Implementações extras poderão receber até **1,0 ponto adicional**, limitado à nota máxima da atividade:

- Dashboard com gráficos (0,25)
- Upload de capa dos livros (0,25)
- Paginação nas consultas (0,25)
- Recuperação de senha (0,25)
- Notificação de atraso (0,25)
- Dockerização do projeto (0,50)
- Deploy online funcional (0,50)
- Testes automatizados (0,50)

**Observação:** Todos os integrantes deverão estar presentes e participar da apresentação. A ausência ou falta de participação poderá impactar a nota individual do aluno.